Panduan Materi Pembelajaran ML Engineering: ALPR On-Device Mobile

Modul / Topik	Fase	Konsep & Teori Mendalam	Rumus Matematis & Algoritma	Implementasi Kode & Sintaks Kunci	Jebakan Umum (Gotchas) & Solusi	Status

1. Geometri Citra & Layout Memori Tensor	Fase 1	• **Koordinat Diskrit 2D**: Origin $(0, 0)$ di sudut kiri-atas; sumbu x horizontal kanan, y vertikal ke bawah.
		• **Spatial Indexing**: Array NumPy memakai baris-kolom `img[y, x]`, berkebalikan dari notasi Cartesius (x, y).
		• **Perbedaan Ruang Warna**: OpenCV membaca BGR secara baku. Model deep learning dan Matplotlib mengekspektasikan RGB. Ketidakcocokan memicu distorsi ekstraksi bobot warna.
		• **Stride Ordering Tensor**: NCHW (*Channel-First*, PyTorch) mengoptimalkan cache GPU CUDA; NHWC (*Channel-Last*, TF/TFLite) optimal pada CPU ARM mobile dan NPU.
			• **Flattening 2D ke 1D Row-Major**:
			$$\text{index}(y, x) = y \times W + x$$
			• **Konversi Luminansi BT.601**:
			$$Y = 0.299R + 0.587G + 0.114B$$
			• **Normalisasi Min-Max**:
			$$I_{\text{norm}} = \frac{I}{255.0} \in [0.0, 1.0]$$
				``
				import cv2, torch
				
				# Konversi BGR -> RGB
				img = cv2.imread(path)
				rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
				
				# Transpose layout NHWC -> NCHW
				t = torch.from_numpy(rgb).permute(2, 0, 1)
				t = t.unsqueeze(0).float() / 255.0
				``
					• **Gotcha**: Plotting citra BGR di `plt.imshow` membuat mobil merah tampak biru.
					• **Solusi**: Bungkus pipeline input dengan helper standardisasi RGB konsisten.
						SELESAI

2. Standarisasi Anotasi COCO vs YOLO	Fase 1 & 2	• **Format COCO JSON**: Hierarki terpusat (`images`, `annotations`, `categories`). BBox berupa piksel absolut top-left $[x_{\min}, y_{\min}, w, h]$.
		• **Format YOLO .txt**: File teks terdesentralisasi per citra. Format: `class_id x_c y_c w_n h_n`.
		• **Independensi Resolusi**: Koordinat ternormalisasi $0.0 - 1.0$ memungkinkan **multiscale training** $(320, 416, 640)$ tanpa perlu menghitung ulang anotasi geometris.
			• **Normalisasi COCO -> YOLO**:
			$$x_c = \frac{x_{\min} + \frac{w}{2}}{W}, \quad y_c = \frac{y_{\min} + \frac{h}{2}}{H}$$
			$$w_n = \frac{w}{W}, \quad h_n = \frac{h}{H}$$
			• **Denormalisasi ke Piksel**:
			$$x_{\min} = (x_c \times W) - \frac{w_n \times W}{2}$$
			$$y_{\min} = (y_c \times H) - \frac{h_n \times H}{2}$$
				``
				def coco_to_yolo(box, W, H):
				  x, y, w, h = [float(v) for v in box]
				  xc = max(0.0, min(1.0, (x + w/2)/W))
				  yc = max(0.0, min(1.0, (y + h/2)/H))
				  wn = max(0.0, min(1.0, w / W))
				  hn = max(0.0, min(1.0, h / H))
				  return f"0 {xc:.6f} {yc:.6f} {wn:.6f} {hn:.6f}"
				``
					• **Gotcha**: Atribut JSON ter-encode string (misal `"1280"`), memicu `TypeError: '>' between int and str`.
					• **Solusi**: Explicit cast `float(v)` sebelum operasi aritmetika batas.
						SELESAI

3. Morfologi Objek & Negative Sampling	Fase 1	• **Aspect Ratio Plat Indonesia**: Rata-rata empiris ≈ 2.97 ($\approx 3:1$). Menjadi acuan desain target dimensi kanvas pengenal karakter.
		• **Relative Area**: Luas bounding box rata-rata 2.01% dari citra penuh. Masuk kategori **small-to-medium object**, rentan hilang pada tahap feature extraction resolusi rendah.
		• **Negative Samples**: Citra jalan/kendaraan tanpa plat nomor dengan file `.txt` kosong untuk menekan false positive pada grille atau bumper.
			• **Uji Integritas Koordinat**:
			$$\text{Valid} \iff (x \geq 0) \wedge (y \geq 0) \wedge (x + w \leq W) \wedge (y + h \leq H)$$
			• **Rasio Luas Spasial**:
			$$A_{\text{rel}} = \frac{w_{\text{bbox}} \times h_{\text{bbox}}}{W_{\text{img}} \times H_{\text{img}}}$$
				``
				# Anotasi file .txt kosong untuk citra negatif
				if len(boxes) == 0:
				  with open(label_path, "w") as f:
				  pass # File kosong menandakan tidak ada objek
				``
					• **Gotcha**: Membuang citra tanpa plat karena dianggap korup.
					• **Solusi**: Pertahankan untuk melatih ketahanan diskriminasi latar belakang.
						SELESAI

4. Pencegahan Data Leakage (Group Splitting)	Fase 2	• **Data Leakage via Augmentasi**: Gambar dari mobil yang sama (hanya beda rotasi/eksposur hasil scraping) tersebar ke Train dan Test set.
		• **Ilusi Evaluasi**: Model sekadar menghafal fitur background mobil (**overfitting**), metrik validasi sangat tinggi palsu namun jeblok saat diuji di dunia nyata.
			• **Himpunan Partisi Bebas Irisan**:
			$$S_{\text{train}} \cap S_{\text{val}} = \varnothing, \quad S_{\text{train}} \cap S_{\text{test}} = \varnothing$$
			• **Rasio Kardinalitas Grup**:
			$$\frac{\lvert G_{\text{train}} \rvert}{\lvert G_{\text{total}} \rvert} \approx 0.80, \quad \frac{\lvert G_{\text{val}} \rvert}{\lvert G_{\text{total}} \rvert} \approx 0.10$$
				``
				def get_group_key(fname):
				  if ".rf." in fname:
				  return fname.split(".rf.")[0]
				  return os.path.splitext(fname)[0]
				
				# Partisi dilakukan pada list(unique_groups)
				``
					• **Gotcha**: Menggunakan fungsi random split naif pada daftar nama file.
					• **Solusi**: Parsing prefix/klaster gambar dan split berdasarkan entitas unik mobil.
						SELESAI

Modul / Topik	Fase	Konsep & Teori Mendalam	Rumus Matematis & Algoritma	Implementasi Kode & Sintaks Kunci	Jebakan Umum (Gotchas) & Solusi	Status
		• **Klaster Bebas Irisan**: Mengharuskan seluruh varian mobil yang sama masuk ke satu subset secara eksklusif.				

5. One-Stage Detection (YOLOv8n) & Prioritas Recall	Fase 3	• **One-Stage vs Two-Stage**: YOLO meregresi koordinat kotak dan probabilitas kelas dalam satu forward pass tanpa Region Proposal Network terpisah.
		• **Arsitektur YOLOv8n**: Memakai backbone C2f dan head anchor-free dengan $\approx 3.2\text{M}$ parameter, ideal untuk eksekusi on-device.
		• **Prioritas Recall**: Pada sistem cascade, detektor adalah gerbang awal. Jika plat terlewat (**False Negative**), OCR tidak terpanggil sama sekali.
			• **Metrik Klasifikasi BBox**:
			$\text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN}$
			• **Intersection over Union**:
			$\text{IoU}(A,B) = \frac{\text{Area}(A \cap B)}{\text{Area}(A \cup B)}$
				`from ultralytics import YOLO`
				
				`model = YOLO("yolov8n.pt")`
				`model.train(`
				`  data="data.yaml",`
				`  epochs=50,`
				`  imgsz=640,`
				`  batch=16,`
				`  fliplr=0.0,  # Kritis: matikan flip teks`
				`  mosaic=1.0,  # Ekstraksi objek kecil`
				`)`
					• **Gotcha**: Mengaktifkan augmentasi flip horizontal (`fliplr=0.5`).
					• **Solusi**: Wajib set `fliplr=0.0` agar teks plat nomor tidak terbalik seperti cermin.
						SELESAI

6. Augmentasi Khusus Objek Kecil	Fase 3	• **Augmentasi Mosaic**: Menggabungkan 4 gambar dengan skala variatif ke satu frame input 640×640. Memaksa detektor mengenali plat nomor pada resolusi lebih kecil.
		• **Dinamika Kamera Mobile**: Variasi skala (`scale=0.5`) dan rotasi (`degrees=5.0`) meniru getaran tangan dan sudut pengambilan foto pengguna ponsel.
		• **Robustness Pencahayaan**: Jitter HSV value (`hsv_v=0.4`) mensimulasikan kondisi terik, mendung, dan malam.
			• **Transformasi Koordinat Affine 2D**:
			$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} s\cos\theta & -s\sin\theta & t_x \\ s\sin\theta & s\cos\theta & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$
			• **Nilai Value HSV**:
			$V = \max(R, G, B)$
				`augment_cfg = {`
				`  "mosaic": 1.0,`
				`  "scale": 0.5,`
				`  "degrees": 5.0,`
				`  "hsv_v": 0.4,`
				`  "fliplr": 0.0,`
				`}`
					• **Gotcha**: Variasi hue berlebihan (`hsv_h > 0.5`) mengubah warna plat hitam menjadi kuning palsu.
					• **Solusi**: Kunci `hsv_h <= 0.015`, pusatkan variasi pada brightness (`hsv_v`).
						SELESAI

7. Rekayasa Data OCR & Domain Gap	Fase 4	• **Domain Gap**: Model OCR yang dilatih hanya pada potongan plat manual sempurna akan gagal saat menerima BBox prediksi detektor yang bergeser atau miring.
		• **Simulated Imperfect Crop**: Melatih OCR dengan campuran: 60% crop ground-truth bersih dan 40% crop simulasi pergeseran koordinat $3-12\%$.
		• **Aspect-Preserving Resize**: Menghindari huruf terdistorsi font-nya akibat kompresi langsung ke 32×128.
			• **Perturbasi Koordinat BBox**:
			$\Delta x \sim \mathcal{U}(\alpha_{\min} w, \alpha_{\max} w), \quad \alpha \in [0.03, 0.12]$
			• **Skala Aspek Proporsional**:
			$s = \min\left(\frac{H_{\text{target}}}{H_{\text{orig}}}, \frac{W_{\text{target}}}{W_{\text{orig}}}\right)$
				`def prep_ocr(crop_gray, th=32, tw=128):`
				`  h, w = crop_gray.shape[:2]`
				`  s = min(th / h, tw / w)`
				`  nw, nh = int(w * s), int(h * s)`
				`  res = cv2.resize(crop_gray, (nw, nh))`
				`  pad = np.zeros((th, tw), dtype=np.float32)`
				`  pad[:nh, :nw] = res / 255.0`
				`  return pad[np.newaxis, ..., np.newaxis]`
					• **Gotcha**: Resize tanpa padding mengubah rasio font plat pendek secara ekstrem.
					• **Solusi**: Terapkan resize proporsional dengan zero-padding sisi kanan/ bawah.
						TERJADWAL

8. Sequence Modeling Bebas RNN & CTC Loss	Fase 5	• **Bebas FlexDelegate**: Menghindari layer RNN/ LSTM yang memicu crash atau ketergantungan library TensorFlow berat di Flutter mobile.
		• **Pure-Conv TCN**: Memakai Stacked Dilated Conv-1D untuk menangkap dependensi sekuens antar-karakter secara paralel dan ramah mobile GPU/NPU.
		• **Asymmetric Pooling**: Menghasilkan timestep $T = 32$ dari input lebar 128 melalui reduksi
			• **Receptive Field Dilated Conv-1D**:
			$\text{RF}_k = \text{RF}_{k-1} + (K-1) \times d, \quad d \in \{1, 2, 4\}$
			• **Fungsi Loss CTC**:
			$\mathcal{L}_{\text{CTC}} = -\ln \sum_{\pi \in \mathcal{B}^{-1}(Y)} P(\pi \text{ mid } X)$
				`class TCNBlock(nn.Module):`
				
				`  def __init__(self, c=256, d=1):`
				`  super().__init__()`
				`  self.conv = nn.Conv1d(`
				`  c,`
				`  c,`
				`  3,`
				`  padding=d,`
				`  dilation=d,`
				`  )`
				`  self.bn = nn.BatchNorm1d(c)`
					• **Gotcha**: Mendefinisikan token blank di indeks terakhir saat training PyTorch, lalu diekspor ke TFLite yang default-nya indeks 0.
					• **Solusi**: Kunci indeks token blank pada index 0 secara konsisten.
						TERJADWAL

Modul / Topik	Fase	Konsep & Teori Mendalam	Rumus Matematis & Algoritma	Implementasi Kode & Sintaks Kunci	Jebakan Umum (Gotchas) & Solusi	Status

		tinggi penuh ($32 \to 1$) dan reduksi lebar 2 kali ($128 \to 32$).		`def forward(self, x): return x + torch.relu(self.bn(self.conv(x)))`
				
				`loss_fn = nn.CTCLoss(blank=0, zero_infinity=True)`		

9. Evaluasi Multi-Tier & Matriks Ambiguitas	Fase 6	• **Multi-Tier Testing**: Mengisolasi kesalahan: Tier 1 (Oracle GT BBox: performa OCR murni), Tier 2 (Simulated Crop: ketahanan terhadap jitter), Tier 3 (Predicted BBox: performa nyata).
		• **Character Error Rate (CER)**: Mengukur rasio jarak edit Levenshtein terhadap total panjang teks asli.
		• **Confusion Matrix 36x36**: Memetakan pasangan karakter berpenampilan mirip: `0`/`O`/`D`, `8`/`B`, `1`/`I`, `5`/`S`.
			• **Character Error Rate**:
			$$\text{CER} = \frac{S + D + I}{N}$$
			• **Exact Match Accuracy**:
			$$\text{EM} = \frac{1}{M}\sum_{j=1}^{M} \mathbb{I}\left(\hat{Y}_j = Y_j\right) \times 100\%$$
				`import editdistance`
				
				`def cer(p, g): d = editdistance.eval(p, g) return d / max(len(g), 1)`
				
				`em = np.mean([p == g for p, g in zip(preds, gts)]) * 100.0`
					• **Gotcha**: Hanya melaporkan akurasi Exact Match biner; plat salah 1 huruf dianggap seburuk plat salah 8 huruf.
					• **Solusi**: Selalu laporkan metrik CER berdampingan dengan Exact Match.
						TERJADWAL

10. Integrasi Cascade & Heuristik Posisi	Fase 7	• **Pipeline Cascade**: Full frame → Deteksi BBox → Margin Expansion → OCR → CTC Decode → Positional Correction → Regex Validation.
		• **Dekomposisi Bottleneck**: Menghitung proporsi kegagalan yang berasal dari detektor (FN) vs kegagalan model OCR (CER).
		• **Heuristik Sintaks Plat Indonesia**: Memanfaatkan struktur 3 zona (Huruf Depan, Angka Tengah, Huruf Belakang) untuk membetulkan karakter ambigu.
			• **Akurasi Cascade End-to-End**:
			$$A_{e2e} = R_{\text{det}} \times R_{ocr\ \text{mid det}}$$
			• **Pola Regex Standar**: `^[A-Z]{1,2}[0-9]{1,4}[A-Z]{1,3}$`
				`# Perbaikan posisi zona tengah (angka)`
				`char_to_num = {"O": "0", "D": "0", "I": "1", "B": "8", "S": "5"}`
				`mid_corrected = "".join([char_to_num.get(c, c) for c in mid_zone])`
					• **Gotcha**: Membuang hasil pembacaan plat jika tidak cocok 100% dengan regex.
					• **Solusi**: Sertakan teks mentah dan flag `is_valid` agar UI Flutter bisa menampilkan hasil fallback.
						TERJADWAL

11. Kuantisasi TFLite & Deployment Flutter	Fase 8	• **Kuantisasi Bobot FP16**: Mengonversi bobot Float32 ke Float16. Memotong ukuran file hingga 50% ($\leq 10\text{MB}$) dengan degradasi akurasi $< 0.5\%$.
		• **Client-Side Standalone NMS**: Menjalankan Non-Maximum Suppression secara native di sisi aplikasi Dart untuk menghindari custom ops C++ TensorFlow pada model.
		• **Budget Latensi Mobile**: Target total end-to-end $\leq 70\text{ms}$ pada GPU/NPU ponsel.
			• **Kuantisasi Presisi IEEE 754**:
			$$\text{FP32 (32-bit)} \to \text{FP16 (16-bit: 1 sign, 5 exp, 10 mantissa)}$$
			• **Evaluasi Paritas Numerik**:
			$$\text{MAE} = \max_i \lvert \hat{y}^{(i)}_{\text{PyTorch}} - \hat{y}^{(i)}_{\text{TFLite}} \rvert < 10^{-2}$$
				`# Ekspor YOLOv8 ke TFLite FP16`
				`model = YOLO("best_detector.pt")`
				`model.export(format="tflite", imgsz=640, half=True)`
					• **Gotcha**: Hanya menguji latensi di prosesor laptop x86 dan mengira latensinya sama di HP ARM.
					• **Solusi**: Lakukan benchmark latensi langsung pada perangkat fisik Android/iOS target.
						TERJADWAL